Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

# STUDENT CASE STUDY EXECUTION REPORT

| | |
|---|---|
| **Department** | FED |
| **Subject** | DSA -2 |
| **Academic year** | I – III Semester(2025-26) |
| **Faculty Name** | Mohammed Razia Alangir Banu |
| **Student Name** | B.Deepak Goud |
| **Roll Number** | 2520080035 |
| **Branch** | AI&DS |
| **Course Outcome** | 2 |

**Case Study Topic : FoodDash Hyderabad — delivery-fee heatmap optimization**

**FoodDash Hyderabad — delivery-fee heatmap optimization**

FoodDash divides Hyderabad into **n = 16 delivery sectors**, indexed from **0 to 15**.
Each sector stores a delivery-demand score, initially set to **2**.

The platform frequently performs two operations:

1. **Range Increment Update**
   Increase delivery-demand score in all sectors within a range [l, r].
2. **Range Maximum Query**
   Find the maximum demand score within a sector interval.

**Current Event Stream**

1. update [2, 8] += 4
   (Tech-park lunch rush)
2. update [6, 13] += 3
   (Evening office logout surge)
3. query max [0, 15]
4. update [1, 5] += 5
   (Rainfall increases local demand)
5. query max [3, 10]

**Initial Sector Values**

All 16 sectors initially contain:

[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]

The segment tree supports:

- Range Add Updates using Lazy Propagation
- Range Maximum Queries

**Questions:**

(a)

Trace the segment tree after the five operations given above. For every update, identify which internal nodes receive lazy markers and which nodes immediately update their maximum value. For each query, show the internal nodes visited and the final answer returned. Use a compact tabular trace instead of redrawing the tree repeatedly.

(b)

Complete the boilerplate updateRange(node, lo, hi, l, r, delta) method below for lazy-propagation range updates. The pushDown() helper is already provided. Fill in the logic for no-overlap, complete-overlap, and partial-overlap cases. Then state the worst-case number of nodes visited per operation as a function of n, and compute it for n = 16.

```
class SegTreeLazy {

    long[] tree;
    long[] lazy;
    int n;

    SegTreeLazy(int n) {
        this.n = n;
        tree = new long[4 * n];
        lazy = new long[4 * n];
    }

    void pushDown(int node) {
        if (lazy[node] != 0) {

            tree[2 * node] += lazy[node];
            lazy[2 * node] += lazy[node];

            tree[2 * node + 1] += lazy[node];
            lazy[2 * node + 1] += lazy[node];

            lazy[node] = 0;
        }
    }

    void updateRange(int node,
                     int lo,
                     int hi,
                     int l,
                     int r,
                     long delta) {
        // TODO
    }
}
```

(c)

A developer suggests replacing the segment tree with a Fenwick Tree (BIT). Explain precisely why a Fenwick Tree is not ideal for this workload. Identify which operation cannot be efficiently supported in O(log n). Also explain what changes would make Fenwick Tree suitable. Finally, explain briefly why Sparse Tables are also unsuitable here.

**Analysis & Implementation:**

**(a) Segment Tree Trace**

Initial sector values:

[2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2]

**Operation Trace**

| Operation | Internal Nodes Updated / Lazy Marked | Result |
|---|---|---|
| **1. update [2,8] += 4** | Lazy markers applied on segments [2,3], [4,7] (full overlap), [8,8]. Max values updated upward through ancestors [0,7] and [0,15]. | — |
| **2. update [6,13] += 3** | Lazy markers on [6,7], [8,11] (full overlap), [12,13]. Ancestor max values recomputed upward. | — |
| **3. query max [0,15]** | Root node [0,15] directly visited since query covers entire array. | Maximum = 9 |
| **4. update [1,5] += 5** | Push-down performed on partially overlapped nodes. New lazy markers stored at [1,1], [2,3], [4,5]. Max values updated upward. | — |
| **5. query max [3,10]** | Traverses nodes covering [3,3], [4,7], [8,10]. Combines subtree maxima. | Maximum = 11 |

**Value Analysis**

After operation 1:

Zones 2..8 become:
2 + 4 = 6

After operation 2:

Zones 6..13 gain +3

Current important values:

- Zone 6 = 2 + 4 + 3 = 9
- Zone 7 = 2 + 4 + 3 = 9
- Zone 8 = 2 + 4 + 3 = 9

Thus:

$$Query\,max\,[0,15] = 9$$

After operation 4:

Zones 1..5 gain +5

Important values:

- Zone 3 = 2 + 4 + 5 = 11
- Zone 4 = 2 + 4 + 5 = 11
- Zone 5 = 2 + 4 + 5 = 11

- Zone 6 = 9
- Zone 7 = 9

Therefore:

$$Query\ max\ [3,10] = 11$$

---

### (b) Lazy Propagation Code

SegTreeLazy.java > Language Support for Java(TM) by Red Hat > SegTreeLazy

```java
import java.util.Arrays;

class SegTreeLazy {
    long[] tree, lazy;

    SegTreeLazy(int n) { tree = new long[4 * n]; lazy = new long[4 * n]; }

    void build(int node, int lo, int hi, long[] a) {
        if (lo == hi) { tree[node] = a[lo]; return; }
        int m = (lo + hi) / 2;
        build(node * 2, lo, m, a);
        build(node * 2 + 1, m + 1, hi, a);
        tree[node] = Math.max(tree[node  * 2], tree[node * 2 + 1]);
    }

    void pushDown(int node) {
        if (lazy[node] == 0) return;
        for (int c = node * 2; c <= node * 2 + 1; c++) {
            tree[c] += lazy[node];
            lazy[c] += lazy[node];
        }
        lazy[node] = 0;
    }

    void updateRange(int node, int lo, int hi, int l, int r, long d) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) { tree[node] += d; lazy[node] += d; return; }
        pushDown(node);
        int m = (lo + hi) / 2;
        updateRange(node * 2, lo, m, l, r, d);
        updateRange(node * 2 + 1, m + 1, hi, l, r, d);
        tree[node] = Math.max(tree[node  * 2], tree[node * 2 + 1]);
    }

    long queryMax(int node, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return Long.MIN_VALUE;
        if (l <= lo && hi <= r) return tree[node];
        pushDown(node);
        int m = (lo + hi) / 2;
        return Math.max(queryMax(node * 2, lo, m, l, r), queryMax(node * 2 + 1, m + 1, hi, l, r));
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        int n = 16;
        long[] arr = new long[n];
        Arrays.fill(arr, val: 2);
        SegTreeLazy st = new SegTreeLazy(n);
```

```
SegTreeLazy.java > Language Support for Java(TM) by Red Hat > SegTreeLazy
 3   class SegTreeLazy {
35       long queryMax(int node, int lo, int hi, int l, int r) {
47
41       }

         Run | Debug | Run main | Debug main
43       public static void main(String[] args) {
44           int n = 16;
45           long[] arr = new long[n];
46           Arrays.fill(arr, val: 2);
47           SegTreeLazy st = new SegTreeLazy(n);
48           st.build(node: 1, lo: 0, n - 1, arr);
89           st.updateRange(node: 1, lo: 0, n - 1, l: 2, r: 8, d: 4);
50           st.updateRange(node: 1, lo: 0, n - 1, l: 6, r: 13, d: 3);
51           System.out.println("Max in [0,15] = "  + st.queryMax(node: 1, lo: 0, n - 1, l: 0, r: 15));
52           st.updateRange(node: 1, lo: 0, n - 1, l: 1, r: 5, d: 5);
53           System.out.println("Max in [3,10] = " + st.queryMax((node: 1, lo: 0, n - 1, l: 3, r: 10));
54       }
55   }
56
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        Filter (eg. text, !excludeText, I...  Code

Max in [0,15] = 9
Max in [3,10] = 11

[Done] exited with code=0 in 0.735 seconds

[Running] cd "d:\CaseStudy Implementation" && javac SegTreeLazy.java && java SegTreeLazy
Max in [0,15] = 9
Max in [3,10] = 11

[Done] exited with code=0 in 0.709 seconds
```

**Complexity Analysis**

Worst-case nodes visited per operation:

$$O(logn)$$

For:

$$n = 16$$

Tree height:

$$log_2(16) = 4$$

At each level, at most 4 partial-overlap nodes are explored.

Thus:

$$4 \times 4 = 16$$

Approximate maximum nodes visited:

$$16$$

---

**(c) Fenwick Tree vs Segment Tree**

A Fenwick Tree (BIT) is not suitable for this workload because it cannot efficiently support:

$$Range\ Update\ +\ Range\ Maximum\ Query$$

in:

$$O(logn)$$

**Why BIT Fails**

Fenwick Trees are designed mainly for:

- Prefix sums
- Range sums
- Point updates
- Range updates with additive operations

However, the operation here requires:

$$Maximum\ over\ a\ range\ after\ multiple\ range\ additions$$

The maximum operation is not invertible, so BIT cannot maintain correct range maxima efficiently after arbitrary updates**.**

**When BIT Would Work**

Fenwick Tree would become suitable if the workload changed to:

- Range Sum Query
- Prefix Sum Query
- Point Query after range additions

For example:

Total delivery demand across sectors
instead of maximum demand sector

Then dual-BIT techniques could support all operations in:

$$O(logn)$$

**Why Sparse Tables are Unsuitable**

Sparse Tables are efficient only for:

$$Static\ Arrays$$

They support:

$$O(1)$$

range maximum queries after preprocessing.

But even a single update requires rebuilding large parts of the table:

$$O(nlogn)$$

Since FoodDash receives frequent updates, rebuild cost becomes extremely expensive compared to the segment tree's:

$$O(logn)$$

update complexity.

Thus, Segment Trees with Lazy Propagation are the best choice for this dynamic workload.

**GitHub Link:https://github.com/bdeepakg-boop/DSA-Project/upload/main**

| **Student Signature** | **Faculty Signature** |
| --- | --- |
| _____ | _____ |